Experiment - 2

Types of network cables

Aim :- To study of different types of network cables.

a) understand different types of network cable

cable type	category	Max data trans	Advt Disadv	Application
UTP	3	10bps	i) Cheaper ii) More prone to EMI	10 Base T-Ethernet
STP	6,6a	10Gbps	i) Shielded ii) Expensive	Used in data centers
SSTP	7	10Gbps	i) Greater effort install	10G Ethernet
Coaxial cable	RG-6 RG-59	10-100 Mbps	i) High bandwidth ii) Size is bulky	TV network
Fibre optic cable	Single mode	100Gbps	i) High speed ii) Expensive	Max dist 100m

b) Make your own ethernet cross over cable

Step 1: To start construction of device, begin by threading shields onto cable.

Step 2: Next, strip approximately 1.5 cm of cable shielding from both ends. The crimping tool has round area.

Step 3: After you will need untangle the wires. There should be four twisted pairs.

Step 4: Once the order is correct, bunch them together in line, and if there are any that stick out further than others, strip them back to create an even level.

Step 5: Next push the cable right in. The notch at the end of plug needs to just over the cable shielding.

Step 6: After the wires are securely sitting inside the plug, insert it into the crimping tool and push down.

Step 7: Lastly repeat for other end wiring diagram B.

Result:- The different types of network cable is successfully studied.

Scanned with OKEN Scanner

Experiment – 3

CISCO packet Tracer

Aim :- To study the packet tracer tool installation and user interface.

a) **UI overview** :

The layout of packet tracer is divided into the several components.

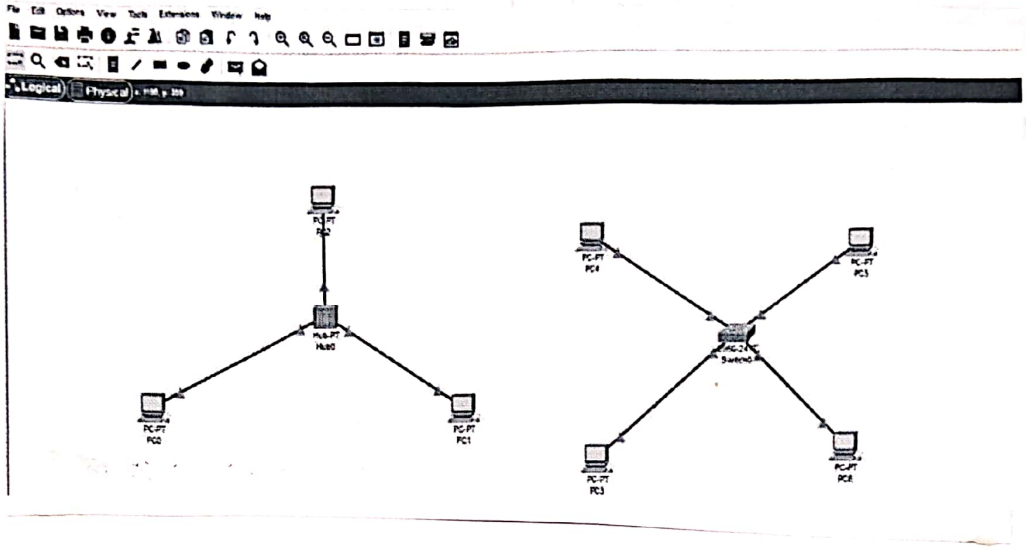
The components are:-

i) menu bar
ii) main toolbar
iii) Logical tabs
iv) workspace
v) common tools bar
vi) Real time tabs
vii) Network component box.
viii) user created packet box

b) Analyze behaviour of network devices using cisco packet tracer simulator.

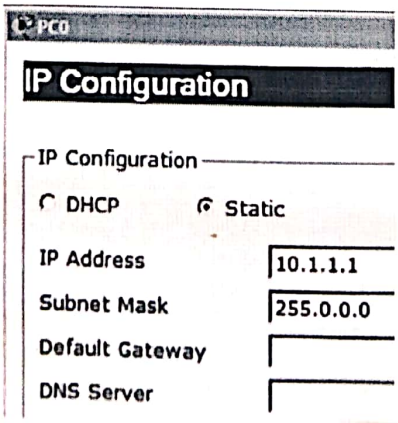
1) from network box drag & drop the component
2) click on connections:-
   a) click on copper straight through cable
   b) Connect 4PCs to switch

Scanned with OKEN Scanner



3) Click on PU, connected to hub, go to desktop tab, click on ip configuration and enter ip address and subnet mask. Here, the default gateway and DNS servers information is not needed as there are only two end devices in network

4) Observe the flow of PDU from source pc to destination pc by selecting the realtime mode.



Scanned with OKEN Scanner

5) Repeat these step 3 to 5.

6) Observe how hub and switch are forwarding the PDU and write your observation

Result:- The cisco packet tracer is successfully studied.

Scanned with OKEN Scanner

Experiment - 4

LAN

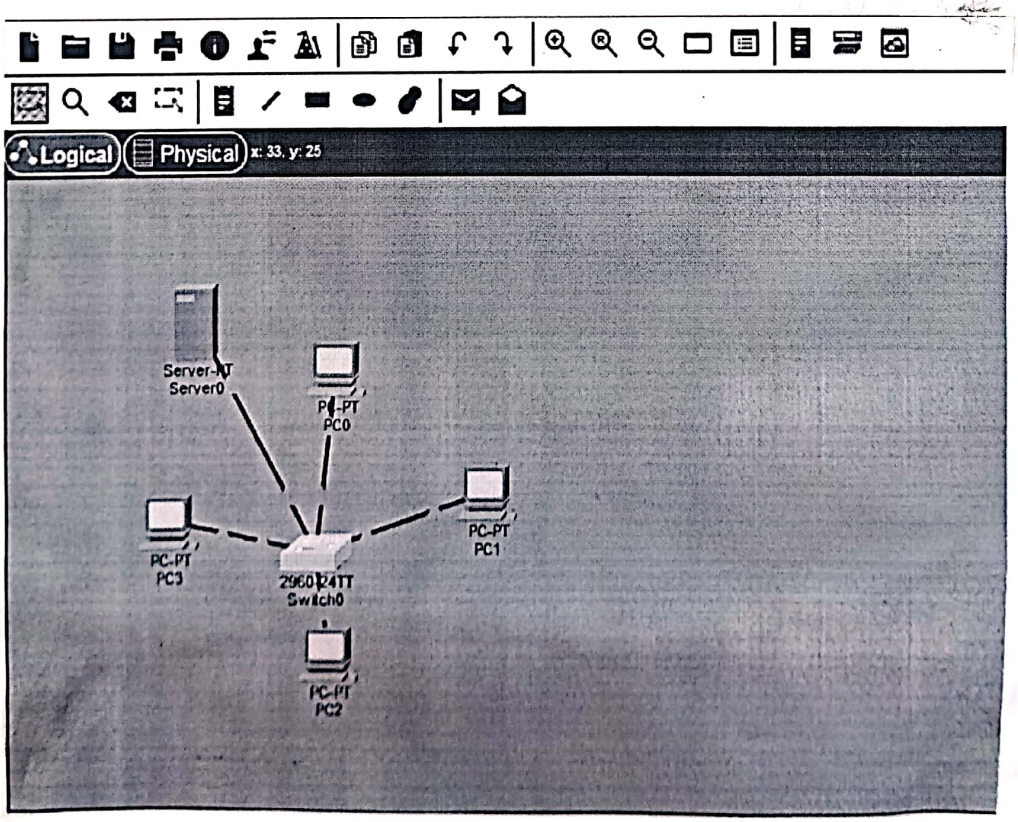
Aim :- To setup and configure a LAN (local area network) using a switch and ethernet cables in your lab.

How to setup a LAN :-

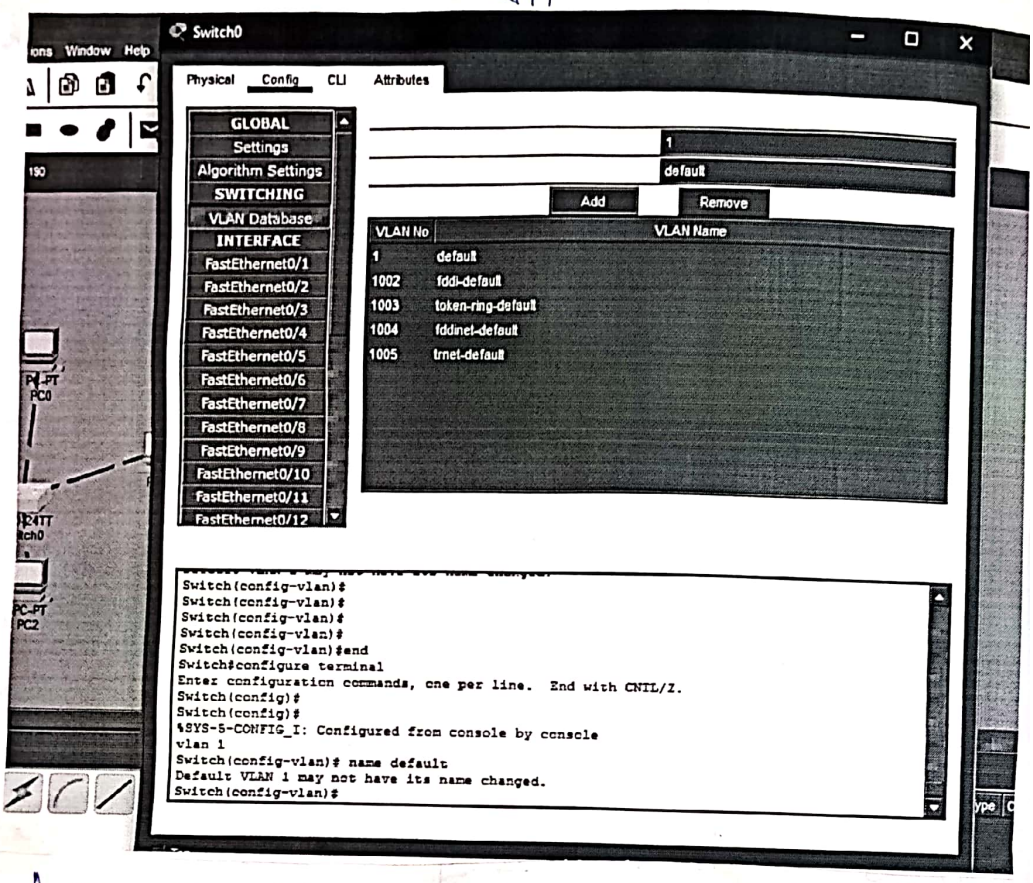
1) Connect your computer to the switch : To access the switches, web interface, you will need to connect your computer to switch.

2) Log in to web interface : open a web browser and enter the IP address of switch in the address bar.

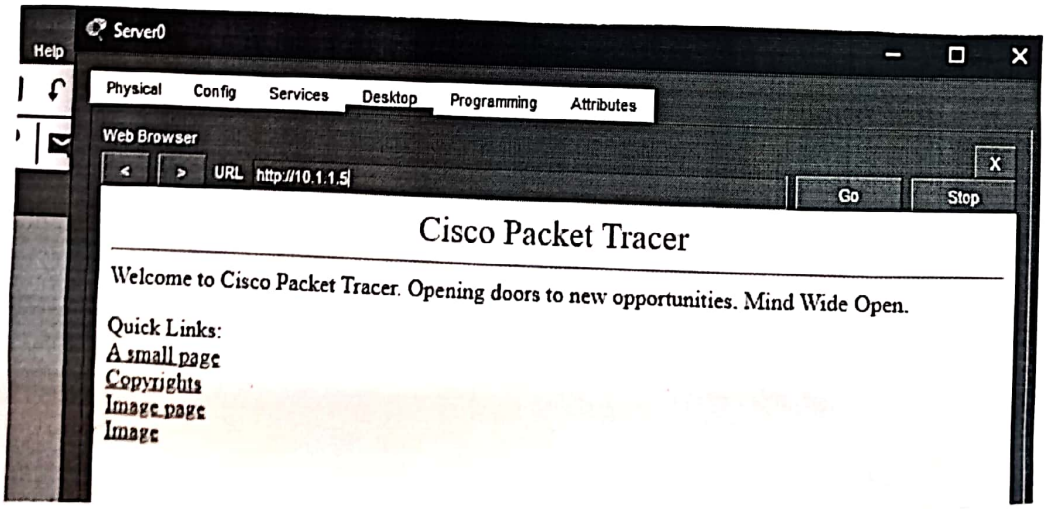
Below is the pictoral representation of the LAN network :-



3) Configure basic settings :- Once you are logged in you will be able to configure.



4) Assign ip address as : 10.1.1.5 subnet mask 255.0.0.0.



Scanned with OKEN Scanner

check the connectivity between switch and other machine by using ping commands in the command prompt of device.

→ select a folder → go to properties → click the sharing tab → share it with everyone on the same LAN.

Result:-

The LAN is configured and set up successfully.

Scanned with OKEN Scanner

Experiment - 5

Experiments on packet capture tool wireshark

Aim :- Experiments on packet capture tool wireshark packet sniffer.

→ sniffs message being sent/received from/by your computer.

→ Store and display the contents of the various protocol fields in the messages

Procedure :-

→ select local area connections in wireshark.

→ Go to capture option

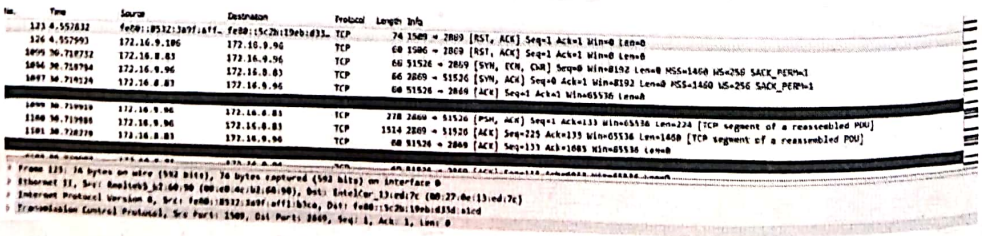
→ Then select stop capture automatically after 100 packets.

→ Then click start capture

→ Save the packets

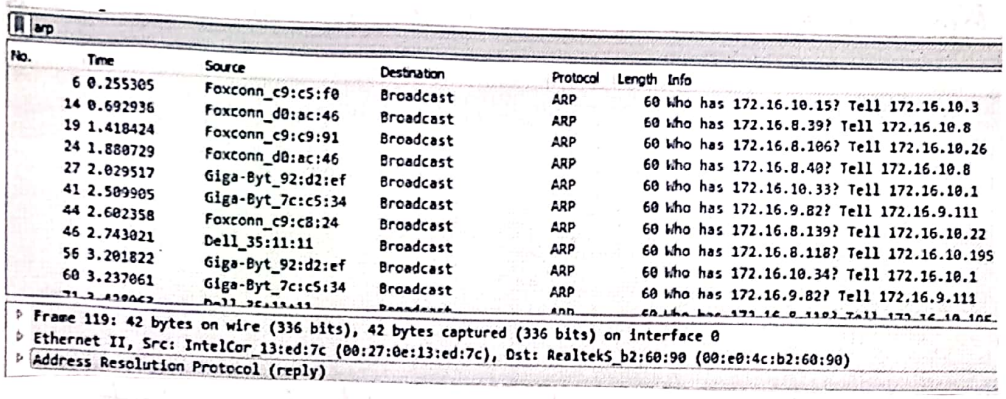
Output :-

1) Create a filter to display only tcp packets > inspect the packet.

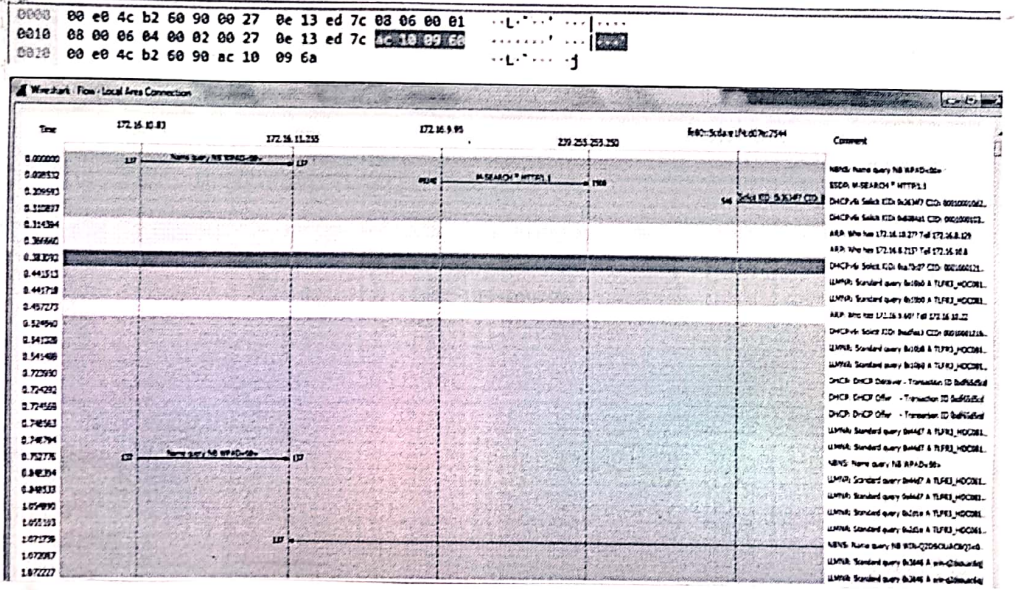


Scanned with OKEN Scanner

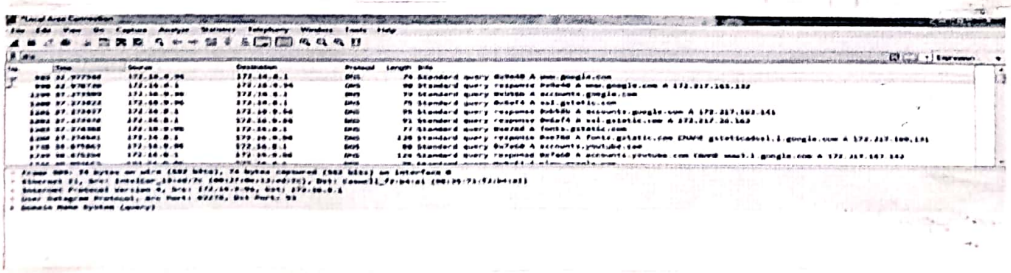
2) Create a filter to display only ARP packet



3) Create a filter to display only DNS packet.



4) Create a filter to display only DHCP packets



Result:- The experiment on wireshark is completed successfully

Scanned with OKEN Scanner

Experiment-6

Hamming-code

Aim:- write a program to implement error detection and correction using hamming code concept.

Procedure:-

→ create a sender program with below features

1) Input to sender file should be a text
2) Apply hamming code concept on binary data
3) Save this output.

→ create a receiver program with below features

1) Receiver program should read the input.
2) Apply hamming code on binary data
3) If there is error display position of error.

Sender code:

```
def calculate_parity_bits(data_bits):
    r=0
    while (2**r) < (data_bits + r + 1):
        r+=1
    return r.

def hamming_encode(data):
    m= len(data)
    r = calculate_parity_bits(m)
```

Scanned with OKEN Scanner

```
total_bits = m+r
hamming = [0] * (total_bits +1)
data_index = 0
for position in range (1, totalbits +1):
if position & (position -1):
    hamming (position) = int( data[data_index])
    data_index+= 1
parity_position =1
while parity_position <= total_bits :
parity =0
for position in range (1, total bits +1):
if position & parity_position :
    parity ^= hamming (position)
encoded = ""
for position in range (1, total_bits +1):
    encoded += str (hamming (position))
return encoded.
with open ("Input.txt", "r") as file :
    text = file.read ()
print ("original text")
print ( text)
```

Scanned with OKEN Scanner

Receiver code :-

```
def hamming_decode(block):
    total_bits = len(block)
    hamming = (0)
for bit in block:
    hamming.append(int(bit))
error_position = 0
parity_position = 1
while parity_position <= total_bits:
parity = 0
for position in range(1, total_bits+1):
    if position & parity_position:
        parity ^= hamming(position)
if position & (position-1):
    data += str(hamming(position))
character = chr(int(data, 2))
return character.
with open("channel.txt", "r") as channel:
    data = channel.read().split()
print("Received data")
result = ""
```

```
for block in data:
    character = hamming_decode(block)
    result += character
print("\n original message after correction")
print(result)
```

Result:- The hamming code experiment is executed successfully & verified.

Scanned with OKEN Scanner

Experiment-7

Aim :- Write a program to implement flow control at data link layer using sliding window protocol

Server program :-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
# define MAX 100
struct frame {
        int frame_no;
        char data;
};
int main() {
struct frame frames[MAX];
int windowsize, n, i, start = 0;
char message[MAX];
FILE *fp;
int ack;
scanf("%d", &windowsize);
scanf("%s", message);
n = strlen(message);
for (i=0; i<; i++) {
    frame(i).frame_no = i+1;
```

Scanned with OKEN Scanner

```
frame(i).data = message(i);
while (start < n) {
    int end = start + windowsize;
    if (end > n) {
        end = n;
    }
    fp = fopen("sender_buffer", "w");
    if (fp == NULL) {
        printf("error");
        return 1;
    }
for(i = start; i < end; i++) {
    fprintf(fp, "%d" %c \n", frame(i).frame(i).
        data);
printf("frame no: %d DATA = %c\n",
        frame(i).frameno, frame(i).data);
fclose(fp);
printf("frames stored in sender buffer");
sleep(5);
fp = fopen("Receiver buffer", "r");
if (fp == NULL) {
    continue;
}
```

```
if (ack > start + 1 && ack <= end) {
    printf("ACK received %d\n", ack);
    start = ack;
    printf("Sliding window moved forward.\n");
}
else if (ack < 0) {
    ack = -ack;
    printf("NACK received for frame %d\n", ack);
    start = ack - 1;
}
printf("All packets transmitted successfully\n");
return 0;
}
```

Receiver program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    FILE *fp, *out;
    int frameNo;
    char data;
    int expected = 1;
    while (1) {
```

Scanned with OKEN Scanner

```
fp= fopen ("Sender_buffer", "r");
if (fp== NULL){
 printf ("Waiting");
sleep(2);
 continue;
}
While(fscanf (fp, "%d %c", &frameNo, &data)==2)
{
 if (frameNo == expected){
  out = fopen ("Receiver_buffer", "w");
 if(out == NULL) {
  fclose (fp);
 return 0;
}
}
fclose (fp);
 return 0;
}
 return 0;
}
```

Result :-

The code has been executed successfully.

Scanned with OKEN Scanner

Experiment - 8(a)

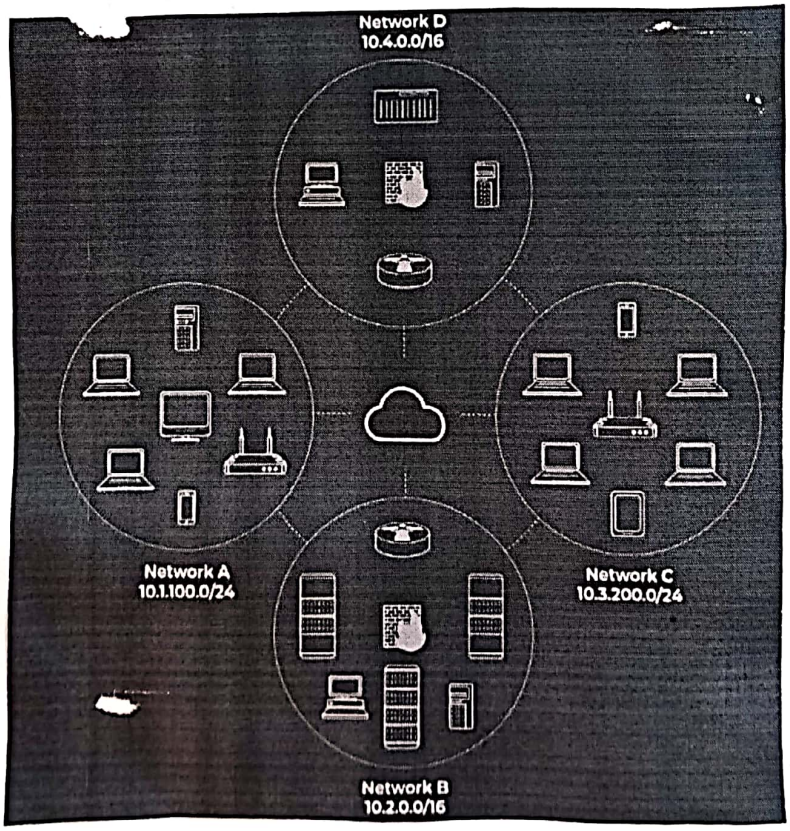
Aim : To discover live host using Nmap scans (ARP, ICMP, TCP / UDP) on tryhackme platform room (Nmap live host discovery)

Introduction :-

When targeting a network we need an efficient tool to handle repetitive tasks. This room is first of four in nmap series.

- Nmap live host discovery
- Nmap basic port scans
- Nmap Advanced port scans
- Nmap port

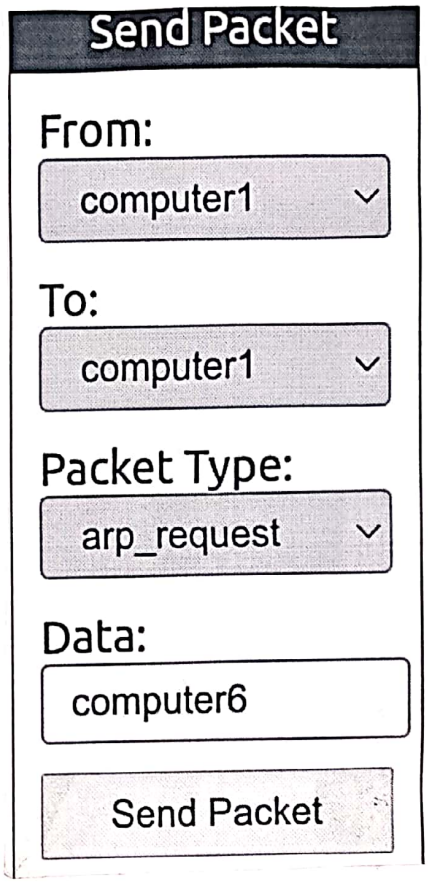
Subnetworks :



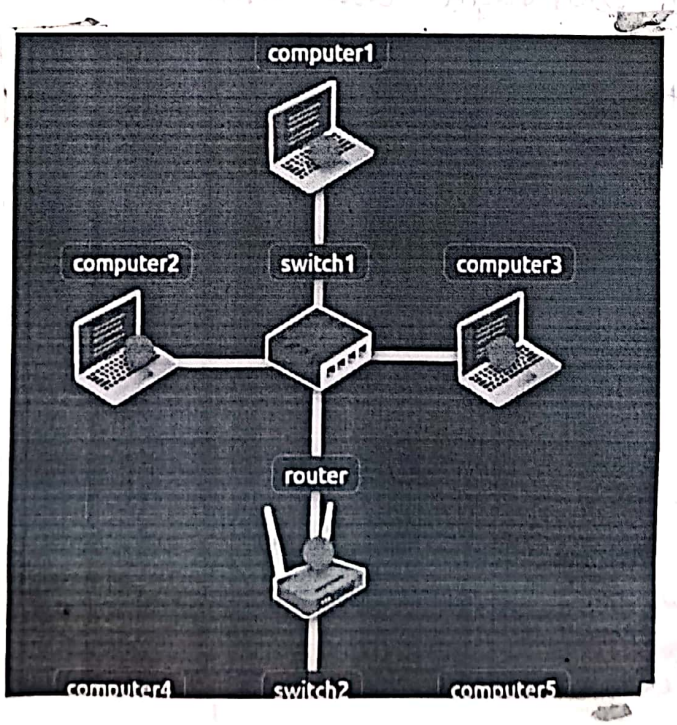
Scanned with OKEN Scanner

This figure display 2 type of subnets :-

* /16 subnets
* /24 subnets

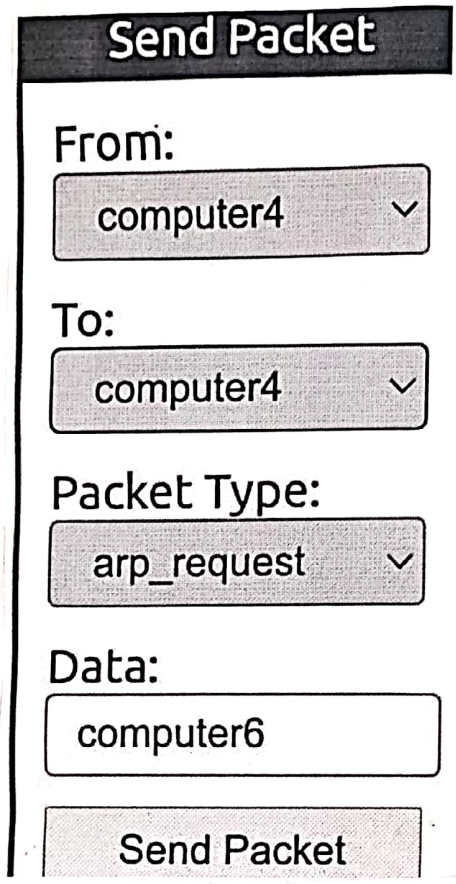


from computer "to computer,
packet type : "ARP request".



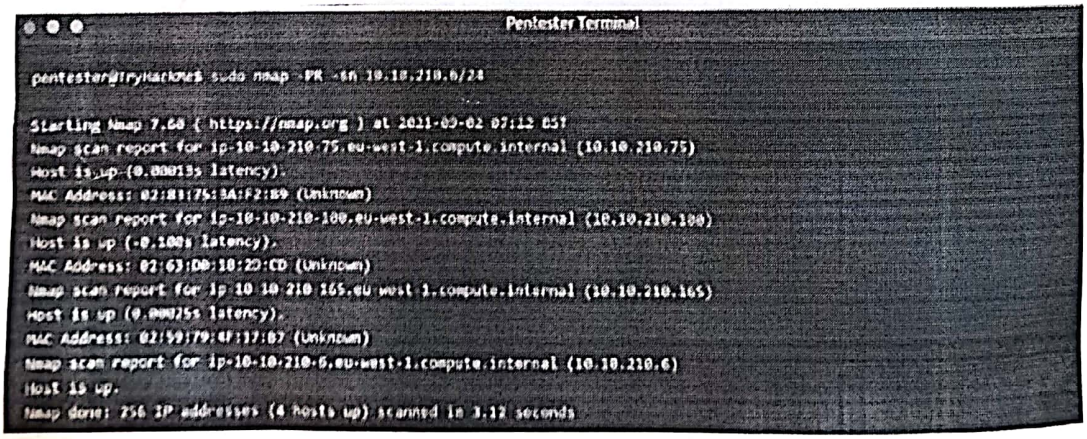
Scanned with OKEN Scanner

- from computer 4 to computer 4
- packet type = "ARP request".



Nmap Host discovery using ARP,

When a privileged user tries to scan target on local network, nmap uses ARP request.



Nmap host discovery using ICMP :-

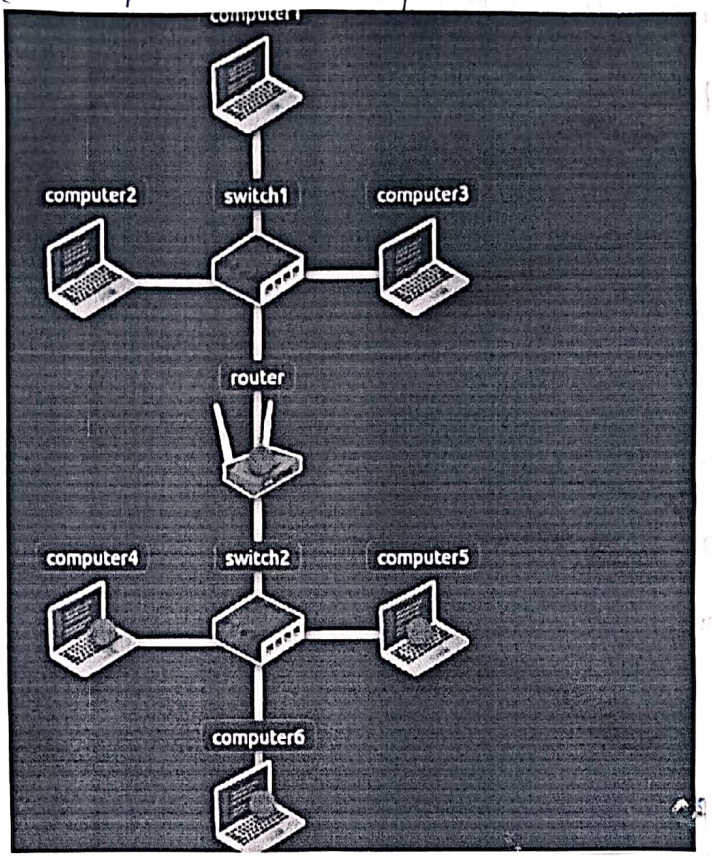
A simple method to identify live hosts on target network is by pinging each IP address and

Scanned with OKEN Scanner

checking for response.

UDP ping:

While sending UDP packets to open UDP ports may not trigger a response, sending them to closed UDP ports.



Result:-

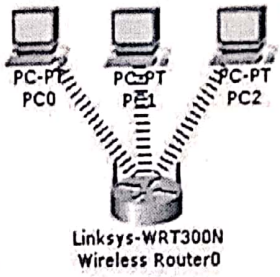
The experiment has been completed fully and got verified.

Scanned with OKEN Scanner

Experiment-8(b)

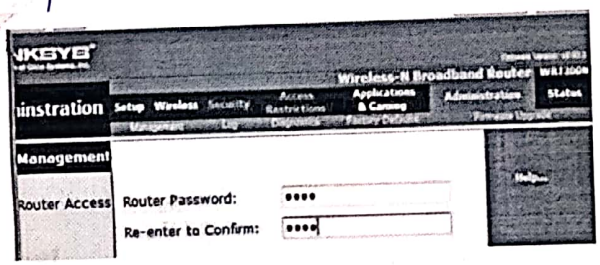
Aim: Configuration of wireless LAN using Cisco packet tracer

Design a topology with three PCs connected from linksys window router.

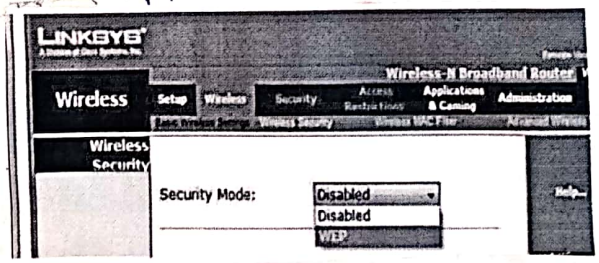


Procedure:-

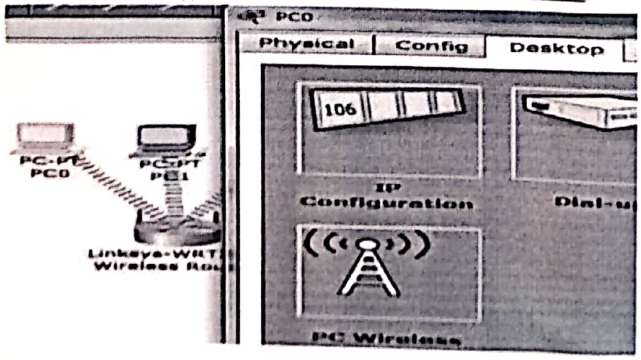
Step 1: Click on wireless router, select admin tab from top menu.



Step 2: Now select wireless security & change security mode to WEP.

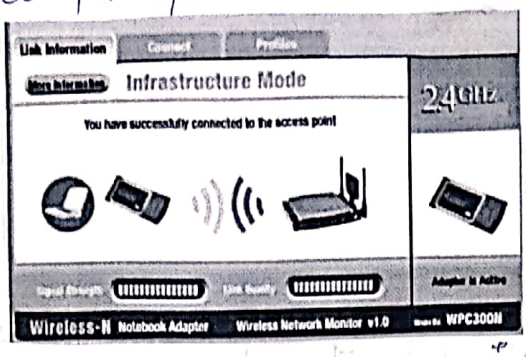


Step 3: Set Key 1 to 0123456789



Scanned with OKEN Scanner

Step4: Connect PC's from window cricket.



Step5: Click on connect tab & click on refresh button

Result:- The configuration has been completed successfully and got verified.

Experiment-9

Aim: Implementation of subnetting in Cisco packet Tracer simulator.

Classless IP subnetting is a technique that allows for more efficient use of IP address by allowing for subnet mask.

Creating a network topology :-

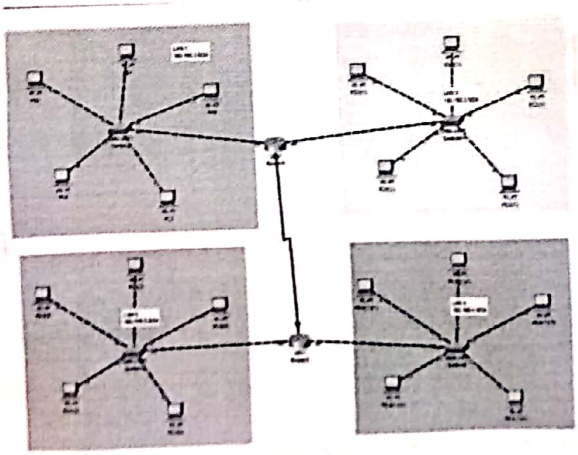
To create a network topology in packet tracer, select "New" button in top, then select network & general.

Adding the devices :

To add a device, select the device from bottom left and drag it onto network topology.

Subnetting:

To subnet the network address of 192.168.1.0/24 to provide enough space for at least 5 address for end device, switch and router.



Configuring the devices:

Right click on routers and select "CLI". This will open by configuring the routers. Right click CLI enter following commands.

```
#enable
# interface fastethernet 0/0
# ip address (ip address) (subnet mask)
# no shutdown
# exit
interface fastethernet 0/1
# ipaddress (ip address) (subnet mask)
no shutdown
exit.
```

Result :- The implementation has been completed successfully and got verified.

Scanned with OKEN Scanner

Experiment-10(a)

Aim: Interworking with routers. In Cisco packet Tracer simulator.

procedure:

Step1: Configure router 1

Step2: Configure PCa

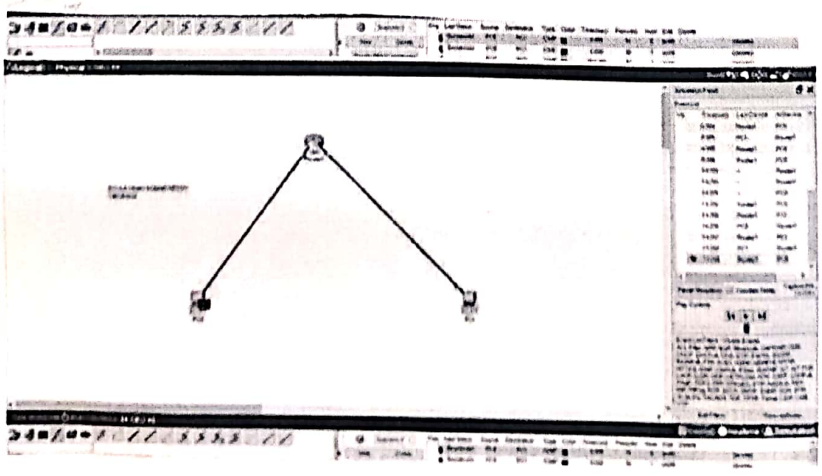
Step3: Connecting PC with Router

Router configuration table:-

Device name	IP address Ethernet 0/0	Subnet mask	IP address ethernet 0/1	Subnet mask
Router	192.168.10.1	255.255.255.0	192.168.20.1	255.255.255.0

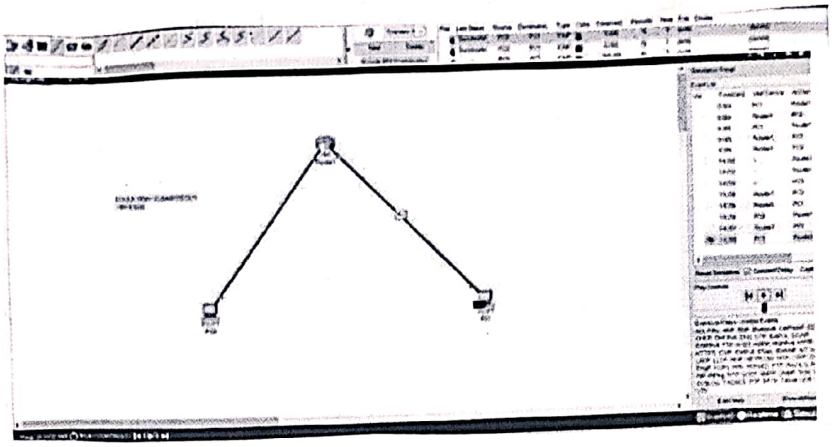
PC configuration table:-

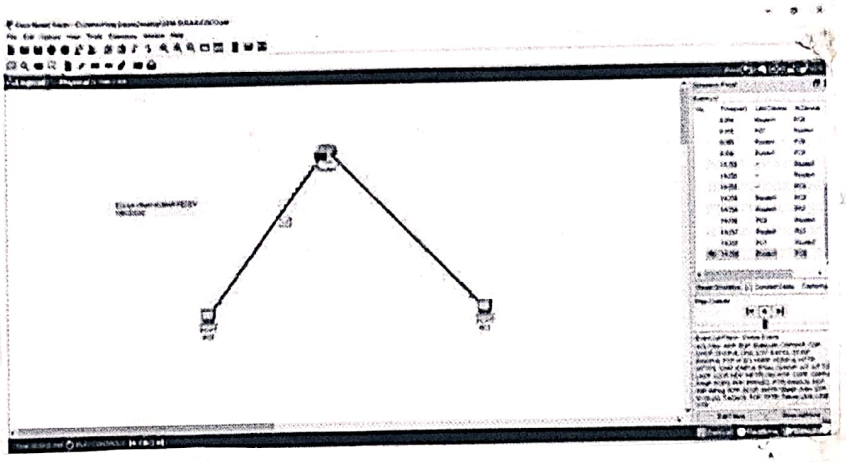
Device	IP address	Subnet mask	Gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1
Sending a PDU: from PC0 to PC1:-			



Scanned with OKEN Scanner

Acknowledgement from PC4 to PC0



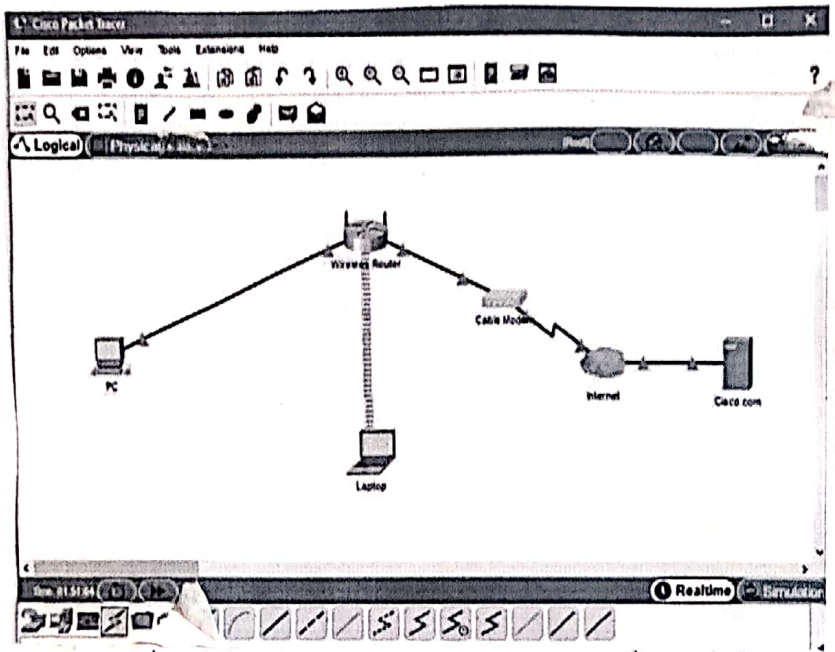


Result :- The internetworking with routers has been completed successfully.

Scanned with OKEN Scanner

Experiment - 10(b)

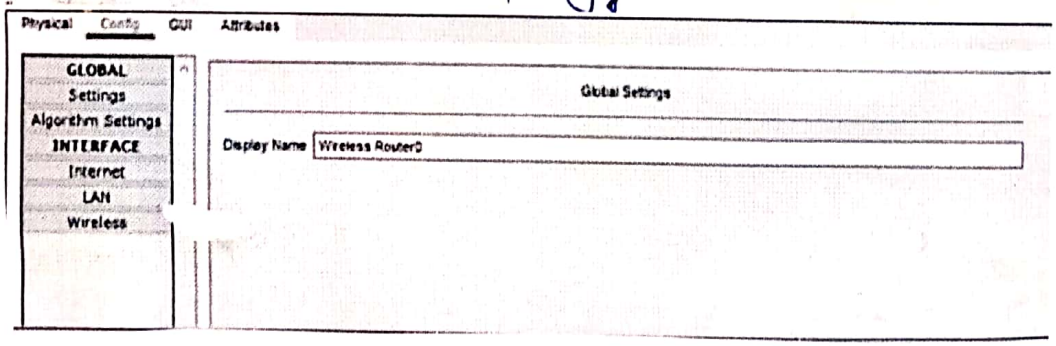
Aim: Design and configure an Internetworking with wireless router, HTTP server and Intrd cloud.



part 1: Build a simple network in the logical topology workspace.

→ Step 1: Launch packet Tracker

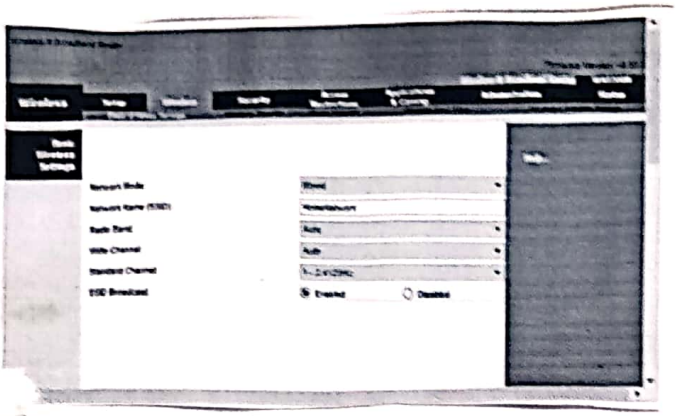
→ Step 2: Build the topology

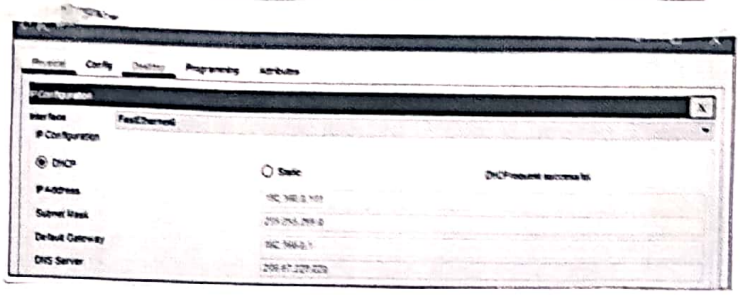


part 2: Configure the network devices

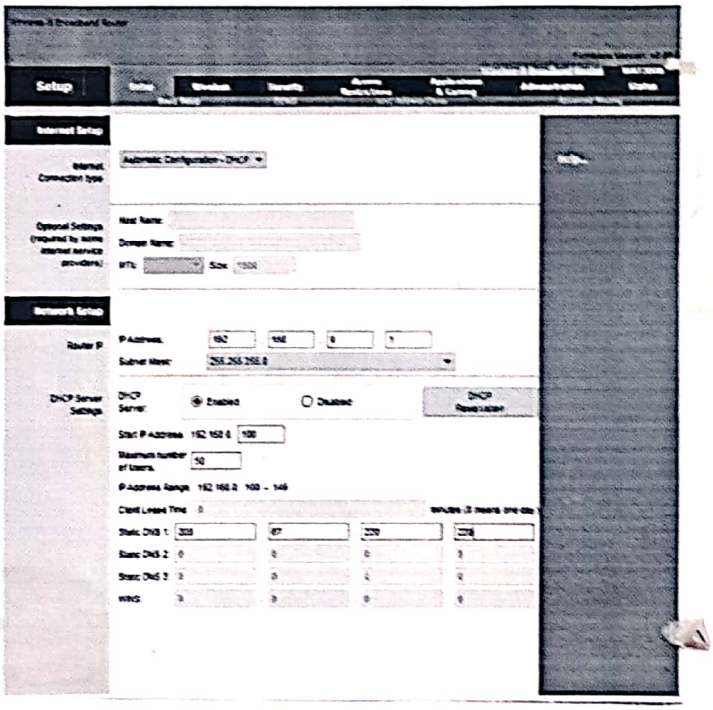
→ Step 1: Configure the wireless router.

→ Step 2: click the OK button and proceed with the configuration

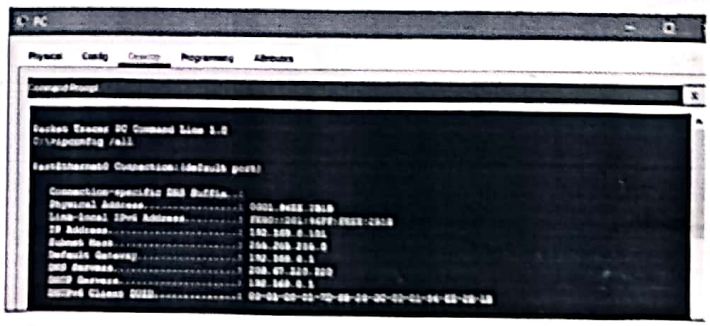




Step 3: Configure the laptop.

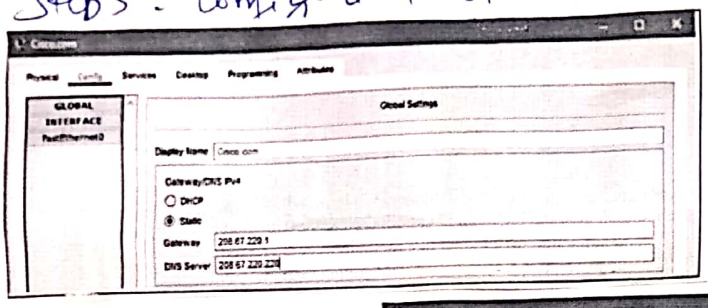


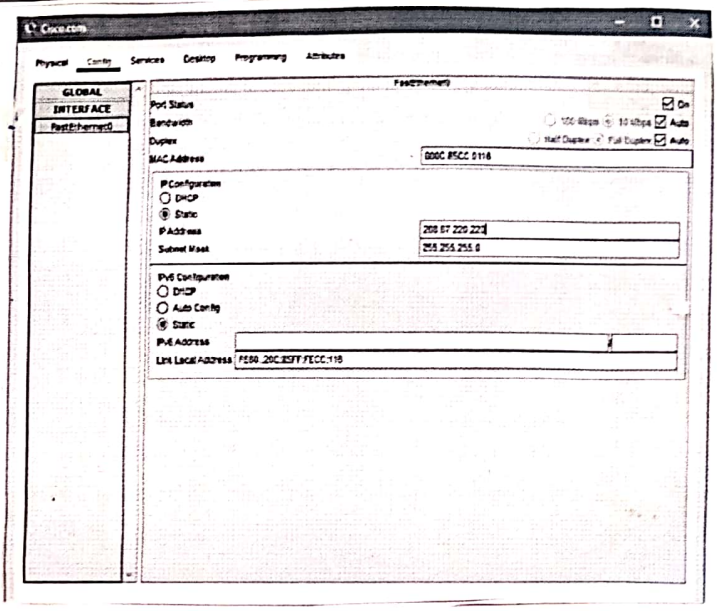
Step 4: configure the PC.



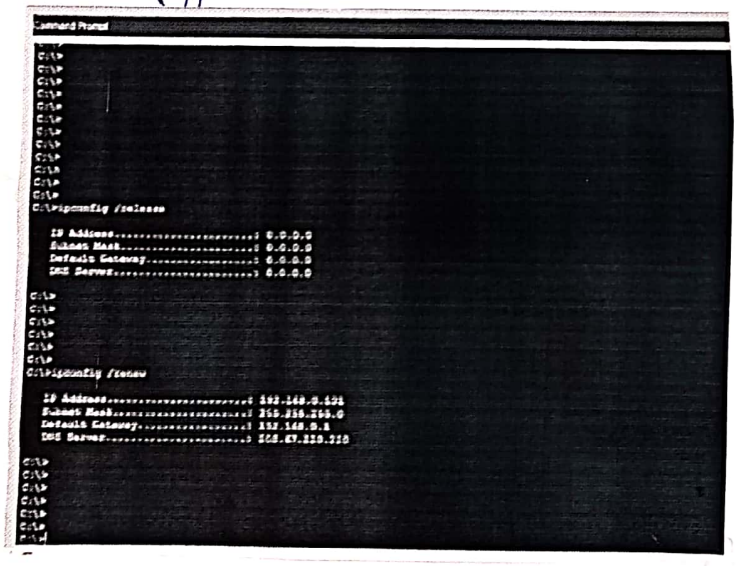
Scanned with OKEN Scanner

Step 5: Configure Internet cloud,





Step-6: Configure the Cisco.com server,



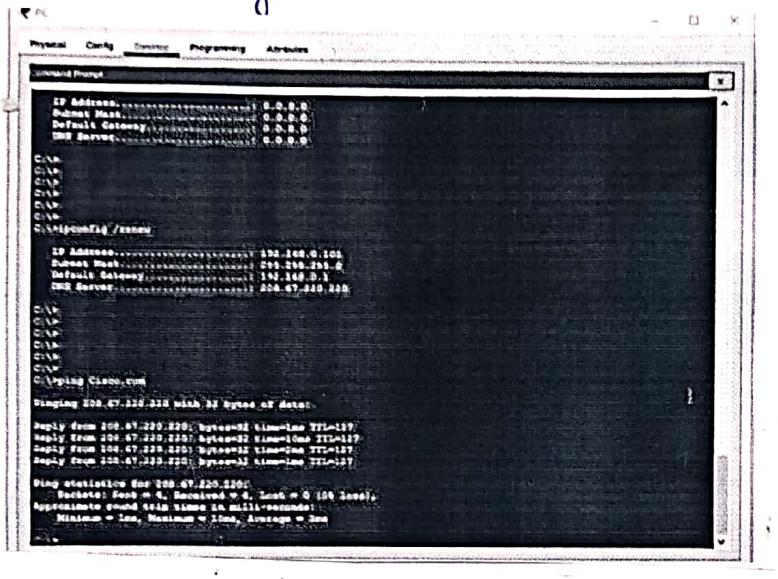
part 3- Verify connectivity

Step 1: Refresh the IPv4 setting on PC

a) verify that PC is receiving IPv4 configuration information from DHCP

Scanned with OKEN Scanner

b) Test connectivity to cisco.com server from PC.



Result :- This experiment has been completed successfully and got verified.

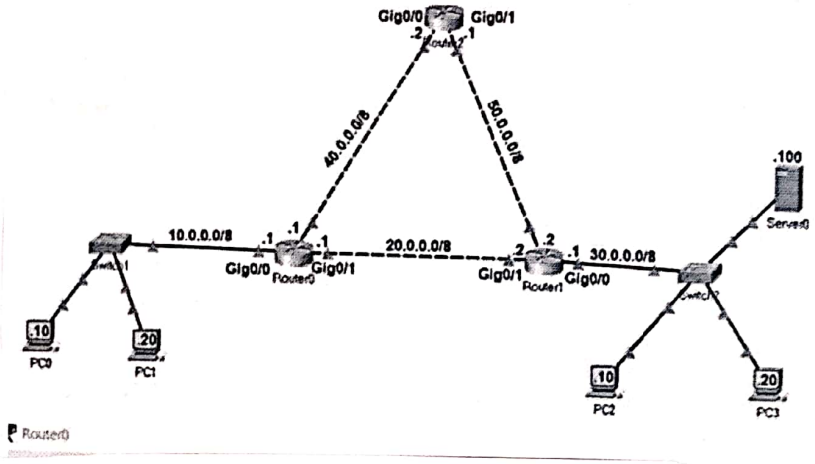
Scanned with OKEN Scanner

Experiment-11(a)

Aim: Simulate static routing configuration using cisco packet tracer.

procedure:

→ Setting up a practice lab.



Creating, adding, verifying static routes:-

Connected network of each router:-

Router	Available on local	Available on other routers.
Router 0	10.0.0.0/8 20.0.0.0/8	30.0.0.0/8, 50.0.0.0/8
Router 1	20.0.0.0/8, 30.0.0.0/8.	10.0.0.0/8 40.0.0.0/8.

Router 0 configuration :-

Router > enable

Router # configure terminal

router (config) # ip route 30.0.0.0 255.0.0.0 20.0.0.2

Router(config)# ip route 30.0.0.0 255.0.0.0 40.0.0.0
2.20

Router(config)# exit.

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10 Primary route
Router(config)#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20 Backup route
Router(config)#ip route 30.0.0.100 255.255.255.255 40.0.0.2 10 Primary route
Router(config)#ip route 30.0.0.100 255.255.255.255 20.0.0.2 20 Backup route
Router(config)#ip route 50.0.0.0 255.0.0.0 40.0.0.2 10 Primary route
Router(config)#ip route 50.0.0.0 255.0.0.0 20.0.0.2 20 Backup route
Router(config)#exit
Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

Router 1 configuration :-

Router > enable

router # configure

terminal

Router(config)# ip route 10.0.0.0 255.0.0.0 20.0.0.10

Router(config)# ip route 10.0.0.0 255.0.0.0 50.0.0.20

Router(config) # exit.

Router2

Physical Config CLI Attributes

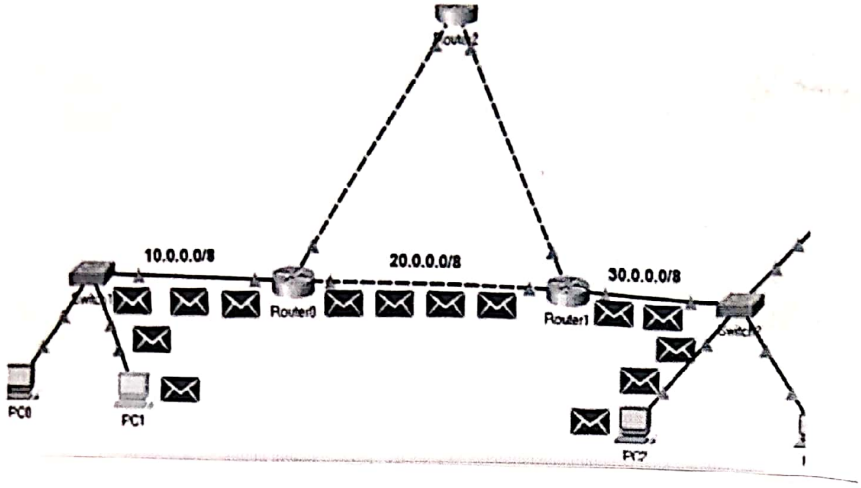
IOS Command Line Interface

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with
Router(config)#ip route 10.0.0.0 255.0.0.0 40.0.0.1
Router(config)#ip route 30.0.0.0 255.0.0.0 50.0.0.2
Router(config)#exit
Router#show ip route static
S    10.0.0.0/8 [1/0] via 40.0.0.1
S    30.0.0.0/8 [1/0] via 50.0.0.2

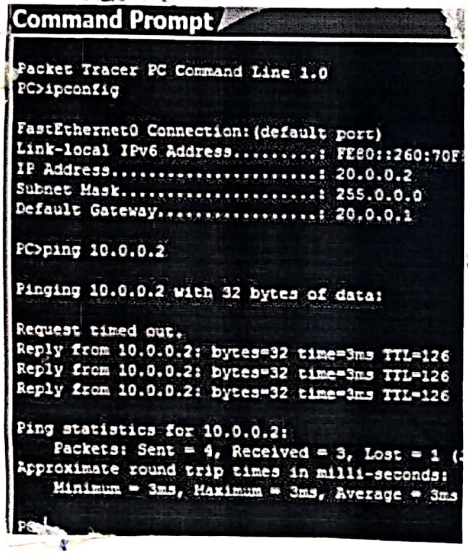
Router#
```

Verifying static routing

Scanned with OKEN Scanner



To simulate the failure of main route, you can delete the link between router 0 & router 1.



Result:- The experiment has been completed successfully.

Scanned with OKEN Scanner

Experiment 11(b)

Aim :- Simulate RIP using Cisco packet tracer.

Initial IP configuration :-

Device	Interface	IP config	Connected unit
PC0	fast Ethernet	10.0.0.2/8	Router 01
Router 0	Fa 0/1	10.0.01/8	PC0's fast ethernet
Router 0	S0/a/1	192.168.1.254/30	Router1 S0/0/1
Router1	S/0/0	192.168.1.259/30	Router 0's S0/0/1
Router 1	S/0/1	192.2.68.1256/30	Router 0's S/0/1
PC1	Fast ethernet	20.0.0.2/30	Router 01.

Assign IP address to PC1 :-

Double click PC1 and click desktop menu item and click IP config...

Router 0 :

Router (config)# interface fast ethernet 0/0

Router (config-if) # ip address 10.0.0.1

255.0.0.0 Router (config)# no shutdown

Router (config)# exit

Router 1

Router > enable

Router #configure

Router (config) #interface serial 0/0/0

Router (config if) # no shutdown

Router (config -if) # exit.

Router 2

Router > enable

Router > configure

terminal

Router (config) # interface fast Ethernet 0/0

Router (config -if) # ip address 20.0.0.1

255.0.0.0 Router (config -if) # no shutdown

Router (config -if) # exit

Access the command prompt of PC and use ping command to test connectivity from PC0.

Result :- The experiment has been completed successfully.

Experiment 12(a)

Aim : Implement echo client server using TCP/UDP sockets

Algorithm :- TCP server

1) Start the program
2) Create server socket using a port number
3) wait using accept()
4) Create I/P & O/P streams
5) Receive message from client.
6) Send same back to client.
7) Stop.

TCP client

1) Start the program
2) Create a socket & connect to server using port no
3) Create I/P & O/P streams
4) Read a message from user
5) Send same to server
6) Receive the echoed message
7) Stop.

I/P :-

Enter message : Hello Server.

Scanned with OKEN Scanner

O/P : Server started
Waiting for client
Client connected
Message from client : Hello server.

Result : The program has been executed successfully and got verified.

Scanned with OKEN Scanner

Experiment (2(b)

Aim : To implement a chat - client server application using Tcp/~~UDP~~ sockets.

Algorithm : Tcp chat server.

1) Start the program
2) Create server socket with port number.
3) Wait using accept()
4) Create ip/op stream
5) Receive message from client
6) Display message
7) send reply to client
8) Close socket & stop program

Tcp chat client

1) Start the program
2) Create a socket & connect to server
3) Create ip/& op stream
4) read a message from client server.
5) send message from client server
6) Display reply
7) Close the socket and stop program.

Scanned with OKEN Scanner

I/P :-

Client : Hello

Server : Hi

Client : How are you?

Server : I am fine.

Client : Exit.

O/P :-

Server started...

Waiting for client

client connected.

Result :- The program has been executed successfully & got verified.

Scanned with OKEN Scanner

Experiment - 13

Aim - Implement your own ping program

Program :-

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
int main(){
    int sock;
    char buffer[1024];
    struct sockaddr_in addr;
    struct icmphdr icmp;
    sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
if (sock<0){
    perror("socket");
return 1;
}
memset(&icmp, 0, sizeof(icmp));
icmp.type = ICMP_ECHO;
icmp.code =0;
sendto(sock, &icmp, sizeof(icmp), 0, (struct
sockaddr*) &addr, size of caddr));
printf("Reply received from %s\n");
```

Result: The program has been executed successfully.

Scanned with OKEN Scanner

Experiment - 14

Aim: Write a code using RAW socket to implement packet sniffing

Program:

```
#include <stdio.h>
#include <unistd.h>
#include <netinet/ip.h>
int main() {
  int sock;
  char buffer[65536];
  struct sockaddr saddr;
  int len = sizeof(saddr);
  while (1) {
  int n = recvfrom(sock, buffer, sizeof(buffer), 0,
&saddr, (socklen_t*) &len);
  if (n > 0) {
  struct iphdr *ip = (struct iphdr *) buffer;
printf("Packet received\n");
}
close(sock);
return 0;
}
```

Result:

The program has been executed successfully.

Scanned with OKEN Scanner

Experiment -15

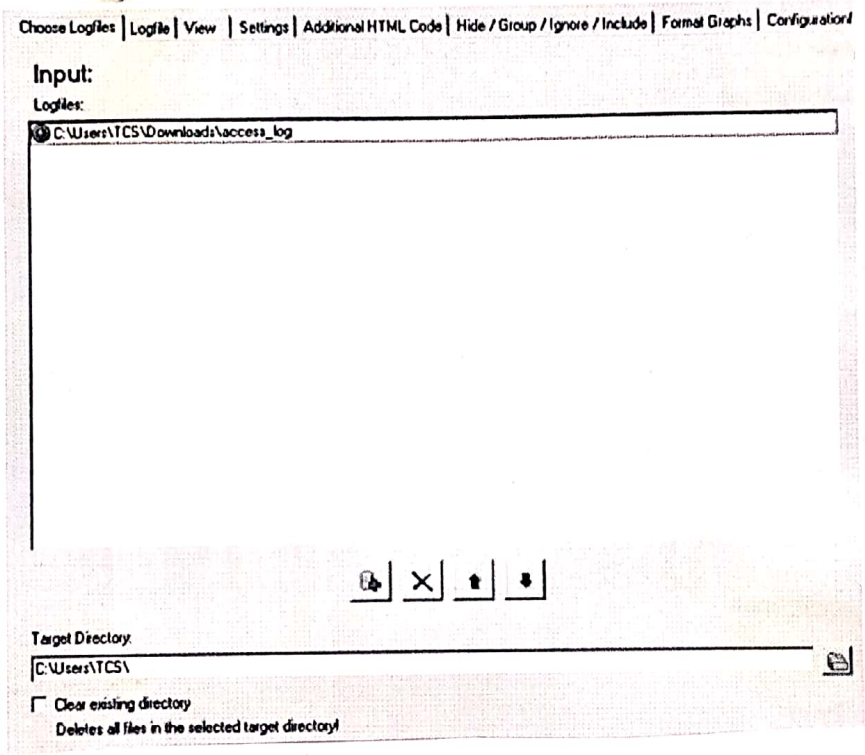
Aim: To analyze the different types of web logs using webalizer tool.

procedure:

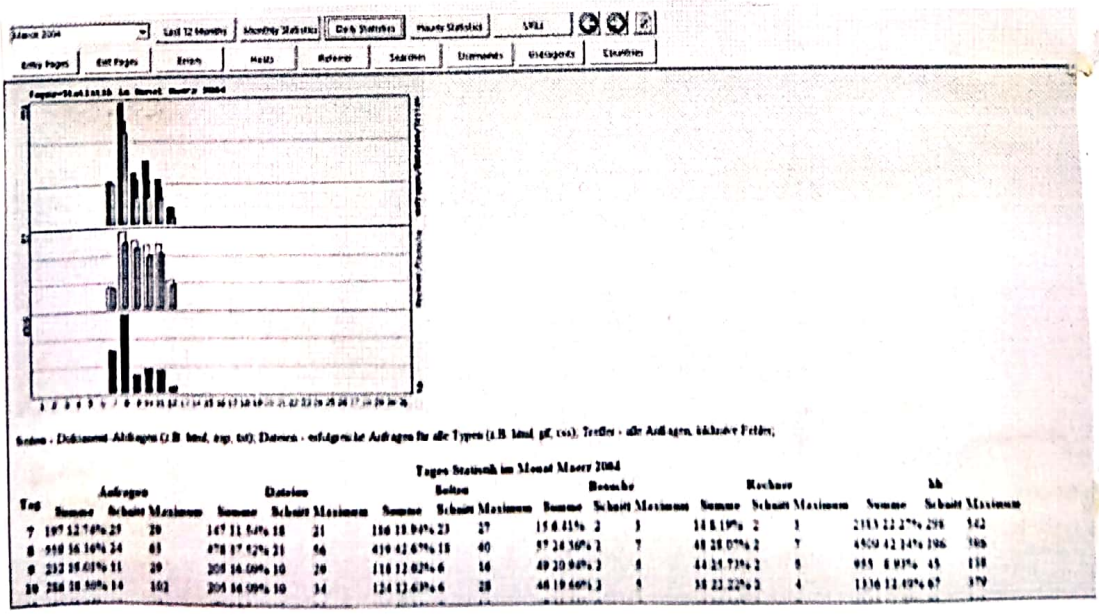
Step 1: Run webalize window version

Step 2: Input web log file.

Step 3: Press run webalizer.



O/p: Monthly Statistics:



Scanned with OKEN Scanner

Hort :-

March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries

Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen		Dateien		Seiten		kb		Besuche		Dauer	Land	Rechnername
1	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85	shawcable.net
2	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62	inktomisearch.com
3	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10	overture.com
4	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70	hevanet.com
5	35	2.26%	29	2.28%	14	1.43%	218	2.03%	7	2.99%	1.31	6.00	bc.ca
6	29	1.88%	28	2.20%	14	1.43%	97	0.91%	1	0.43%	3.43	3.43	panduit.com
7	23	1.49%	14	1.10%	10	1.02%	135	1.26%	1	0.43%	12.35	12.35	geovariances.fr
8	22	1.42%	22	1.73%	8	0.81%	67	0.63%	1	0.43%	3.72	3.72	cox.net
9	19	1.23%	19	1.49%	7	0.71%	61	0.57%	1	0.43%	1.52	1.52	ac.il
10	19	1.23%	11	0.86%	10	1.02%	51	0.48%	2	0.85%	3.04	5.87	netinfo.bg
11	15	0.97%	14	1.10%	13	1.32%	130	1.22%	3	1.28%	1.29	3.87	e-Inet
12	13	0.84%	13	1.02%	13	1.32%	120	1.12%	1	0.43%	3.88	3.88	telia.net
13	13	0.84%	13	1.02%	1	0.10%	28	0.26%	1	0.43%	0.13	0.13	net.ar
14	13	0.84%	13	1.02%	2	0.20%	30	0.28%	1	0.43%	0.87	0.87	dhl.com
15	12	0.78%	11	0.86%	9	0.92%	68	0.63%	1	0.43%	25.53	25.53	tiscali.de
16	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.33	0.33	wtbts.org
17	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.10	0.10	qwest.net
18	11	0.71%	11	0.86%	7	0.71%	41	0.38%	2	0.85%	0.56	1.03	radiant.net
19	10	0.65%	9	0.71%	7	0.71%	41	0.38%	1	0.43%	1.40	1.40	net.au
20	10	0.65%	10	0.78%	2	0.20%	61	0.57%	1	0.43%	0.17	0.17	3_343_It_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen		Dateien		Seiten		kb		Besuche		Dauer	Land	Rechnername
1	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10	overture.com
2	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85	shawcable.net
3	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70	hevanet.com

Result : The experiment has been completed successfully and got verified.

Scanned with OKEN Scanner